

DM-2026-010

Bruce Dombrowski
Systems Engineer, Payload Software Integration

DATE: July 23, 2026

TO: PCFPlan Stakeholders, ISS Payload Integration

SUBJECT: PCFPlan Increment Editor: Display-Window Selection and Payload-Config Copy

OFFICE OF PRIMARY RESPONSIBILITY: Payload Software Integration (PSI)

1. **Purpose**

This memorandum documents two design decisions for the PCFPlan increment editor, arising from issue #30 (GitLab #20, reported by K. Larson): (a) how the four displayed increments are selected, and (b) how a payload-configuration copy between increments resolves collisions. Both are captured as proposed requirements pending baseline: REQ-011a (issue #32) and REQ-060 (issue #33).

2. **Background**

PCFPlan displays four ISS increments (timeline and table columns) selected by a date-driven window — the current increment (by today's date) plus the next three (REQ-011, `pdf_generator.py:_select_increments`). The user cannot choose which four are shown. It was verified against the local database that deactivating the current increment shifts the window's left edge *backward* to the previous still-active increment (an older one), which is the mechanism behind the unexpected columns reported in #30. Separately, a newly created increment's payload configuration starts empty, forcing manual rebuild; the reporter requested the ability to copy a prior increment as a starting point.

3. **Decision 1 — Increment Display-Window Selection (REQ-011a, #32)**

Options considered:

- **1A — Starting-increment selector (SELECTED).** The user picks a starting increment; the four displayed are that increment plus the next three.
- **1B — As-of date override.** The user sets an effective date; the window stays date-driven but anchored to that date.
- **1C — Pin four increments individually.** The user selects each of the four columns explicitly.

Decision: Option 1A. The user selects the starting increment; the displayed four are that increment plus the next three. The selection is **sticky** — persisted in the database (per the project convention that all editable state lives in the database, not config files) so it

survives session to session. Absent an explicit selection, the system retains the default REQ-011 behavior (current increment plus next three).

Rationale: 1A is the simplest mental model and matches how the user reasoned about the problem. It reuses the existing “anchor plus next three” windowing with a user-chosen anchor instead of today’s date, minimizing UI and code change. 1B and 1C add complexity (date semantics; four-way selection state) without additional value for the stated need.

4. Decision 2 — Payload-Config Copy Collision Behavior (REQ-060, #33)

Context: Copying a source increment’s items into a target increment can collide with the items table constraint `UNIQUE(increment, module_key, parent_name, item_name)`. A collision occurs only when the target already contains an item with the same module, parent, and name. In the primary use case the target is empty, so no collision arises.

Options considered:

- **2A — Skip duplicates.** Keep the target’s existing item; ignore the incoming copy.
- **2B — Overwrite, warn if not empty (SELECTED).** Source items replace colliding target items; if the target already contains items, warn the user (confirmation) before proceeding.
- **2C — Empty-target-only.** Refuse the copy (warn) if the target is not empty.

Decision: Option 2B (recommended, pending user confirmation). On copy, if the target increment already contains items, the system warns the user before proceeding; on confirmation, source items overwrite colliding target items, and non-colliding target items are unaffected. Because this behavior directly affects the reporter’s day-to-day workflow, her preference is being solicited on issue #30 before REQ-060 is baselined; the decision will be finalized on her input and is explicitly held open to revision.

Rationale: Overwrite makes “copy I77 → I78” an idempotent “make the target match the source” operation with predictable results. The not-empty warning guards against accidental clobbering of manual edits. Skip (2A) was rejected because it can silently leave a half-updated target that diverges from the source in non-obvious ways.

5. Implementation and Constraints

- Sticky window selection persisted in the database; stdlib-only; CUI-safe; database auto-pushed on quit per project workflow.
- Window selection updates both the timeline and the table columns (REQ-021, which requires the table columns to match the four shown).
- Copy carries all item attributes (type/style, flags, note references, arrival/descent, hierarchy).
- Regression tests added per REQ-QA-001 so the defect class cannot recur.

6. Verification

- **REQ-011a (Test):** selecting a starting increment renders exactly those four columns in timeline and table; with no selection, behavior matches REQ-011.

- **REQ-060 (Test):** copying I77 → I78 reproduces I77's items under I78; edits to the copy do not affect the source; collisions overwrite; a non-empty target triggers the warning.
- Both mapped in the verification matrix (requirement → file → line → method) per SE Phase 4.

7. References

- Issues #30 (report), #32 (REQ-011a), #33 (REQ-060), #31 (hardening).
- PCFPlan requirements: REQ-011, REQ-021, REQ-026, REQ-031, REQ-032, REQ-QA-001.
- Systems Engineering process, Phase 3 (Decision Documentation); RFC 2119.